

Architectural Awareness
Taxi Tap by Git It Done



Contents

1	Introduction	3
2	Strong Software Architecture	3
2.1	System Overview	3
2.2	Architectural Style	4
3	Design Patterns and Architectural Patterns	4
4	Quality and Scalability Requirements	5
5	Testing Framework and Quality Assurance	5
5.1	Automated Unit and Integration Testing	5
5.2	Performance and Scalability Testing	5
5.3	Security Testing	5
5.4	Convex-Specific Testing Constraints	6
6	Coding Standards and Continuous Integration	6
7	Deployment Model	6
8	Conclusion	7

1 Introduction

Taxi Tap by Git It Done is a mobile application designed to modernise the South African minibus taxi industry without replacing its fundamental system. Our architecture focuses on scalability, modularity, and maintainability through the use of a service-oriented, event-driven design. This document highlights how our team demonstrated architectural excellence through system design, the use of design patterns, robust testing practices, and coding standards that ensure software quality and long-term scalability.



2 Strong Software Architecture

2.1 System Overview

Taxi Tap is a mobile application designed to revolutionise South Africa's minibus taxi industry. We achieve this by digitising route information, creating a semi-structured booking system while preserving the flexibility that makes taxis an essential transport mode. Our system connects passengers and drivers through a location-aware application that facilitates taxi requests, communicates driver locations, manages payments, and provides real-time vehicle tracking - all without changing the fundamental system.

Our app has two separate interfaces, a passenger interface and a driver interface. We use GPS-based proximity alerts to notify passengers of their approaching taxi. We also include real-time taxi tracking showing vehicle location and available seats. We provide a driver rating and feedback system for passengers and also improve safety by having a passenger activate their ride with a pin provided by the driver. We provide users with cash payment support and features where both passengers and drivers can see information of current or past rides. Our system supports multiple languages and also supports multi-leg journeys, where a passenger can book a ride with up to two different routes to

get to their destination.

Our app does not aim to replace the taxi system, but rather to enhance it.

2.2 Architectural Style

Taxi Tap follows an **Event-Driven and Service-Oriented Architecture (SOA)** that promotes loose coupling, real-time responsiveness, and scalability. The system uses asynchronous event communication between components to handle ride requests, location updates, and notifications efficiently. Each service - such as Booking, Authentication, Payment, and Location - is structured as an independent module communicating through well-defined APIs.

A key architectural feature of our system is Convex's **function-based API generation**. Each backend function automatically becomes an accessible API endpoint through the Convex framework. This eliminates manual endpoint creation and ensures consistent function exposure, parameter validation, and type safety across the system.

Our frontend and backend are connected through a cloud-based deployment model using Convex's serverless backend. This allows modular function execution, strong type safety, and seamless scaling. The architectural style supports our main goals of real-time interaction, low latency, and easy scalability.

3 Design Patterns and Architectural Patterns

Taxi Tap employs several software design patterns that enhance modularity, maintainability, and reusability across the system:

- **Observer Pattern:** Used in the Notification System to automatically notify users (drivers and passengers) of state changes such as route updates, proximity alerts, and booking confirmations. This ensures efficient communication without direct coupling between components.
- **Mediator Pattern:** Implemented in the Taxi Request System to manage complex communication between drivers and passengers. Instead of interacting directly, both communicate through a central mediator that ensures proper data exchange and consistency.
- **Model-View-ViewModel (MVVM):** Adopted on the frontend (React Native) to separate the UI (View) from business logic (ViewModel) and data (Model). This structure allows better testability and maintainability of the mobile application.
- **Leader-Follower (Replication):** Integrated into Convex's infrastructure to replicate data across multiple zones, improving system availability and fault tolerance.

These patterns collectively contribute to a cleaner architecture, simplifying testing, parallel development, and system evolution.

4 Quality and Scalability Requirements

Taxi Tap’s architecture was designed around five top quality attributes: **availability, scalability, usability, performance, and security.**

- **Availability:** Convex’s replication and automatic failover ensure a 99.5% uptime, even during failures. The serverless design allows the system to continue running smoothly with minimal manual intervention.
- **Scalability:** The backend scales horizontally through Convex’s automatic function execution and data sharding. This supports over 100 concurrent ride requests with consistent sub-300ms response times during performance tests.
- **Usability:** The MVVM architecture, combined with responsive UI design and offline support, ensures that users with varying digital literacy can easily navigate the app.
- **Performance:** Non-critical tasks such as notifications and analytics run asynchronously in the background, ensuring that core user actions — like ride requests and location updates — respond within milliseconds.
- **Security:** All data is encrypted in transit and at rest. Role-Based Access Control (RBAC) is enforced at the function level, and user authentication is handled through secure, token-based sessions.

5 Testing Framework and Quality Assurance

Our testing policies and workflow ensure that Taxi Tap meets functional and non-functional requirements consistently.

5.1 Automated Unit and Integration Testing

We use **Jest** for both unit and integration testing. Tests are automated through our GitHub Actions CI pipeline. Code is prevented from merging into the main branch unless all tests pass successfully. We maintain a minimum coverage of 80% across the project.

5.2 Performance and Scalability Testing

Performance and scalability are tested through Node.js-based frameworks and the K6 load testing suite. The system is validated to sustain 100+ concurrent users, 4+ requests per second per endpoint, and a P95 response time below 300ms.

5.3 Security Testing

A Python-based security testing suite is used to detect vulnerabilities including SQL injection, XSS, CORS misconfigurations, and authentication bypass attempts. Continuous security monitoring ensures that no high-severity vulnerabilities remain undetected.

5.4 Convex-Specific Testing Constraints

Because Convex automatically generates APIs for each function, traditional REST testing tools are not compatible. Our testing policy adapted by implementing a custom **HTTP wrapper service** that translates test requests into Convex function calls. We account for validation-related errors caused by synthetic data and interpret performance metrics as end-to-end results, including wrapper overhead.

6 Coding Standards and Continuous Integration

Our team follows strict coding and documentation standards to maintain quality, clarity, and collaboration efficiency.

- **Language and Framework:** TypeScript and JavaScript using React Native and Convex.
- **Linting and Formatting:** ESLint and Prettier ensure consistency across all codebases.
- **Code Style:** 2-space indentation, 100-character line limit, single quotes, and mandatory semicolons.
- **Testing Standards:** All modules must include unit and integration tests before merging.
- **Pull Request Guidelines:** Code must pass all linting, testing, and peer reviews before merging.
- **Continuous Integration:** GitHub Actions automate testing, linting, and deployment workflows.

These standards ensure code reliability, readability, and maintainability while supporting efficient collaboration among developers.

7 Deployment Model

Taxi Tap follows a cloud-based deployment model to ensure scalability and availability:

- Frontend is deployed using Expo Go for rapid testing and updates.
- Backend and database are hosted on Convex Cloud, leveraging serverless functions for modular execution.
- Continuous Integration and Deployment are handled automatically through GitHub Actions.

This deployment model ensures that each new feature or fix passes through automated quality gates before being released to production, maintaining reliability and consistency.

8 Conclusion

Our team demonstrated architectural awareness through thoughtful system design, adherence to software engineering principles, and the application of scalable, secure, and maintainable technologies. By leveraging Convex’s automatic API generation, event-driven communication, and robust testing strategies, Taxi Tap achieves both technical excellence and real-world scalability - fulfilling the goals of the **Architectural Awareness Prize**.